Prática de Laboratório: Mensageria com Java e ActiveMQ

1. O Cenário: Plataforma de Mídia "Breaking News"

Imagine o sistema de publicação de um grande portal de notícias. Quando um jornalista termina de escrever uma matéria urgente (Breaking News) e clica em "Publicar", o sistema precisa realizar várias tarefas:

1. Salvar o texto no banco de dados.
2. Gerar as miniaturas (thumbnails) das imagens anexadas.
3. Disparar notificações *Push* para os celulares dos assinantes.
4. Enviar a matéria para o motor de busca (como o Elasticsearch) para indexação.

Se o sistema do jornalista for **síncrono**, ele ficará com a tela travada esperando o redimensionamento das imagens e o disparo de milhares de alertas.

A Solução Assíncrona (EDA)

Neste laboratório, vamos construir o núcleo dessa arquitetura. O sistema do jornalista (Produtor) apenas salvará o texto e enviará um evento chamado NoticiaPublicada para o **ActiveMQ**. Imediatamente, a tela é liberada. Um serviço em *background* (Consumidor) lerá esse evento e simulará o processamento pesado.

---

2. Passo a Passo da Implementação

Passo 1: Subindo a Infraestrutura (Docker)

Antes de escrever qualquer código Java, precisamos do intermediário rodando. Crie um arquivo docker-compose.yml na raiz do seu workspace.

YAML

```yaml
version: '3.8'
services:
  activemq:
```

```
  image: rmohr/activemq:latest
  container_name: broker-midia
  ports:
    - "61616:61616" # Porta de comunicação TCP (JMS)
    - "8161:8161"   # Painel Web de Administração
  environment:
    - ACTIVEMQ_ADMIN_LOGIN=admin
    - ACTIVEMQ_ADMIN_PASSWORD=admin
```

No terminal, execute: docker-compose up -d.

Passo 2: Criando o Projeto Spring Boot

Acesse o Spring Initializr e crie um projeto com as dependências:

- **Spring Web** (Para criar o endpoint REST do jornalista).
- **Spring for Apache ActiveMQ** (Para a mensageria).

No arquivo application.properties, configure a conexão com o broker:

Properties

```
# Aponta para o container Docker
spring.activemq.broker-url=tcp://localhost:61616
spring.activemq.user=admin
spring.activemq.password=admin

# Desativa o broker embutido para forçar o uso do Docker
spring.activemq.pool.enabled=false
```

Passo 3: Configurando o Conversor Universal (JSON)

Para que a mensagem da notícia não viaje como um binário ilegível, configuramos o Jackson para transformá-la em JSON. Crie a classe ActiveMQConfig:

Java

```java
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jms.support.converter.MappingJackson2MessageConverter;
import org.springframework.jms.support.converter.MessageConverter;
import org.springframework.jms.support.converter.MessageType;

@Configuration
public class ActiveMQConfig {

    @Bean
    public MessageConverter jacksonJmsMessageConverter() {
        MappingJackson2MessageConverter converter = new
MappingJackson2MessageConverter();
        converter.setTargetType(MessageType.TEXT); // Garante que será salvo como texto puro
        converter.setTypeIdPropertyName("_type"); // Identificador da classe Java
        return converter;
    }
}
```

Passo 4: O Objeto de Transferência (DTO)

Este é o "pacote" que viajará pela rede. Crie a classe NoticiaDTO:

Java

```java
import java.io.Serializable;

public class NoticiaDTO implements Serializable {
    private String idMateria;
    private String titulo;
    private String autor;
    private boolean urgente;

    // Construtores, Getters e Setters
    public NoticiaDTO(String idMateria, String titulo, String autor, boolean urgente) {
        this.idMateria = idMateria;
        this.titulo = titulo;
        this.autor = autor;
```

```
        this.urgente = urgente;
    }

    // ... (insira os getters e setters aqui)
}
```

Passo 5: O Produtor (O Sistema do Jornalista)

Crie o serviço que irá postar a notícia na fila.

Java

```
import org.springframework.jms.core.JmsTemplate;
import org.springframework.stereotype.Service;

@Service
public class RedacaoProducer {

    private final JmsTemplate jmsTemplate;

    public RedacaoProducer(JmsTemplate jmsTemplate) {
        this.jmsTemplate = jmsTemplate;
    }

    public void publicarNoticia(NoticiaDTO noticia) {
        // Envia o JSON para a fila chamada "queue.noticias.processamento"
        jmsTemplate.convertAndSend("queue.noticias.processamento", noticia);
        System.out.println("[REDAÇÃO] Matéria enviada para processamento assíncrono: " +
noticia.getTitulo());
    }
}
```

Passo 6: O Consumidor (O Worker de Processamento)

Este componente roda em *background*, escutando a fila e agindo assim que a notícia chega.

Java

```java
import org.springframework.jms.annotation.JmsListener;
import org.springframework.stereotype.Component;

@Component
public class ProcessamentoNoticiaWorker {

    @JmsListener(destination = "queue.noticias.processamento")
    public void processar(NoticiaDTO noticia) {
        System.out.println("\n[WORKER] Iniciando tarefas pesadas para a matéria: " +
noticia.getIdMateria());

        try {
            // Simulando tarefas demoradas (I/O)
            System.out.println("-> Gerando thumbnails das imagens...");
            Thread.sleep(2000);

            System.out.println("-> Enviando matéria para o Elasticsearch...");
            Thread.sleep(1500);

            if(noticia.isUrgente()) {
                System.out.println("-> 🚨 DISPARANDO PUSH NOTIFICATIONS (Matéria Urgente)!");
            }

            System.out.println("[WORKER] Processamento finalizado com sucesso!\n");

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.err.println("Erro durante o processamento da matéria.");
        }
    }
}
```

Passo 7: O Gatilho de Teste (Controller)

Por fim, crie uma API REST para que você possa testar o sistema disparando o evento.

Java

```java
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/noticias")
public class NoticiaController {

    private final RedacaoProducer producer;

    public NoticiaController(RedacaoProducer producer) {
        this.producer = producer;
    }

    @PostMapping("/publicar")
    public ResponseEntity<String> simularPublicacao(@RequestBody NoticiaDTO noticia) {
        producer.publicarNoticia(noticia);
        return ResponseEntity.ok("Matéria salva! O processamento das mídias e notificações está
rodando em background.");
    }
}
```

---

3. Como testar o cenário

1. Iniciar a aplicação Spring Boot.
2. Abrir o Postman (ou usar cURL) e fazer um POST para http://localhost:8080/api/noticias/publicar com o JSON:
   JSON
   ```json
   {
     "idMateria": "PUB-994",
     "titulo": "Plantão: Acontecimento Importante na Capital",
     "autor": "Equipe de Jornalismo",
     "urgente": true
   }
   ```
3. **Observar o log do console**: O aluno verá a requisição HTTP terminar imediatamente (liberando o Postman), enquanto os logs do [WORKER] continuarão imprimindo o processamento nos 3 a 4 segundos seguintes.

4. **Verificar o Broker**: Acessar http://localhost:8161/admin e conferir na aba "Queues" que a mensagem passou com sucesso.